

Spec :- UT2 Question Bank + Assignment No: 02

- ✓ 1] Explain Macro Expansion.
- ✓ 2] Explain the working of Two pass Macro processor with neat flowchart and databayer.
- ✓ 3] What are the different functions of loader?
- ✓ 4] Explain the design of Absolute loader
- 5] What Explain design of dynamic linking loader along with the example.
- 6] What is the need of Intermediate Code generation? Explain any two Intermediate code generation ^{forms} with eg?
- 7] What are the phases of Compiler? Give working of each phase for the following statements :-

```
int a, b, c = 1;
a = a * b - 5 * 3 / c;
```
- ✓ 8] Explain different Code optimization techniques with example.
- ✓ 9] Explain different issues in code generation.
- ✓ 10] Write short note on :- Operator precedence parsing.

15/4/26

* Answers for UML Ans:-

Q1 Explain Macro Expansion.

- Macro Expansion is the process in which a Macro call in a program is replaced by the corresponding Macro defn (body) by the Macroprocessor.
- A Macro is a named set of Instⁿ that can be reused Multiple times.
- During expansion, the macroprocessor substitute the actual parameters in place of formal parameters and generates the final Expanded Code.

• Steps of Macro Expansion:

1. The Macroprocessor scans the source program.
2. When a macro call is encountered, it searches for its defn, in the Macro Name Table (MNT).
3. The Macro body is obtained from Macro Defn Table (MDT).
4. Actual arguments are substituted using Argument List Array (ALA).
5. The Expanded code represents the Macro call in the program.

Eg.) macro defn:

```
MACRO INCR A
ADD A,1
MEND
```

macro call:

```
INCR X
expanded code
ADD X,1
```

• Adv

- Reduces code repetition
- Improves readability
- Saves programming time

• Disadv

- Increase prog. size after expansion
- Debugging becomes difficult.

Q2 Write short note on: Operator Precedence Parsing.

- Operator precedence Parsing is a bottom-up parsing technique used in compiler to parse expression based on the precedence of operator.
- It is mainly used for:
 - Arithmetic expressions
 - Handling operator like +, -, *, /
- Basic idea

- Each operator is assigned a ~~precedence~~ precedence level.
- Parser decides which operⁿ to perform first.
- Uses Relⁿ like

Rel ⁿ	Meaning
>	lower precedence
<	higher
=	equal

• Working Principle:

Parser uses a stack & i/p buffer:

Steps:

1. Compare precedence of stack top operator & i/p symbols
2. If i/p has higher precedence → shift
3. If stack has higher precedence → reduce
4. Continue until i/p is accepted

Eg. Expression

$$a + b * c$$

Precedence

$$* > +$$

parsing: $b * c$ evaluated first
than $a + (b * c)$

5. Operator precedence Table (eg)

	+	*	id	\$
+	>	<	<	<
-	>	<	<	<
id	>	>	-	>
\$	<	<	<	=

- Adv.
- Simple to implement
- Efficient for expressing parsing.
- No need for full grammar.

• Limitations

- Works only for operator grammar.
- Cannot handle complex grammar etc.
- Difficult for unary operators sometimes.

Q7 Explain different issues in Code generation.

- Code generation is the final phase of a Compiler where Intermediate Code is converted into target machine code.
- While generating code, the compiler faces several design & implementation issues.
- Major Issues in Code generation: -

1. Input to Code generator
Issue:

- IR is Intermediate Representation (IR)
- Must be accurate & complete

Explanation:

- Code Generation depends on IR + Symbol Table.
- Poor IR $\rightarrow$ Inefficient target code.

2. Target Machine Characteristics

Issue:

- Diff. machines have diff:
 - Instruction sets
 - Registers
 - Addressing modes

Explanation:

- Code must be generated according to target machine

Eg., RISC $\rightarrow$ simple instruction
CISC $\rightarrow$ complex instruction

3. Memory mgmt

Issue: Efficient use of memory location

Explanation:

- Decide
 - Where variables are stored (stack/heap)
 - Temporary variable allocations

• Poor memory use $\rightarrow$ slower execution

4. Register Allocation & Assignment

Issue:

- Limited number of CPU registers

Explanation:

- Decide which variables go into registers
- When to store / load from memory.

Good allocation $\rightarrow$ Faster code.

5. Instⁿ Selection

Issue:

- Choosing best Machine Instⁿ

Explanation:

Same Operⁿ can be implemented in diff. ways

Eg., $a = b + c$

can be

MOV R1, b

ADD R1, C

OR optimized Instⁿ

ADD b, c

choose efficient Instⁿ

6. Order of Evaluation

Issue:

- Order in which expression are evaluated

Explⁿ:

- Affects register usage & performance

Eg., $a + b * c$

Evaluate $b * c$ first

7. Handling of Control-flow

Issue:

- Managing loops, condⁿ, jumps

Explanation:

- Generate correct branching Instⁿ:

Eg.,

if (a > b)

$\rightarrow$ Generate

CMP a, b

JG Label

8. Code Optimization vs code generation

Issue:

- Balance b/w

- Fast code

- Small code

Explanation:

- Sometimes faster code uses more memory
- Trade off is req^d.

Q7
 $\rightarrow$

Explain different Code Optimization Techniques with ex

- Code optimization is the phase of Compiler design where the intermediate code is improved to:
 - Reduce execution time
 - Reduce memory usage
 - Improve efficiency.
- Output remains same, but performance improves

- Types of Code Optimization Techniques :-

1. Constant folding

Idea: Compute constant expression at compile time

Eg.,

before: $a = 5 * 3 + b$

after: $a = 15 + b$

2. Constant Propagation

Idea: Replace variable having constant values

Eg., $x = 10$

Before $y = x + 5$

after $y = 10 + 5$

Can further apply constant folding $\rightarrow y = 15$

3. Common subexpression Elimination

Idea: Avoid recomputing same expression multiple times

Eg., Before $t_1 = a * b$

$t_2 = a * b + c$

After $t_1 = a * b$

$t_2 = t_1 + c$

4. Dead Code elimination

Idea: Remove code that is never used

Eg., Before $x = 10$

$y = 20$

return x

After $x = 10$

return x

$y = 20$ is removed.

5. Strength Reduction

Idea: Replace costly operⁿ with cheaper ones.

Eg., Before $x = y * 2$

After $x = y \ll 1$

6. Copy Propagation

Idea: Replace variable copies with original value.

Eg., Before $a = b$

$c = a + d$

After $c = b + d$

7. Loop Optimizⁿ

Improve performance inside loops

Eg., Before

for ($i = 0; i < n; i++$) {

$x = a * b$

$y = x + 1$

}

After

$x = a * b$

for ($i = 0; i < n; i++$) {

$y = x + 1$

}

Q7] What are the different functⁿ of loaders?

- A loader is a system that loads an object program into Memory, prepare it for execution and starts its execution.
- Functions of Loaders :-

1. Allocation

- The loader allocates Memory space to the program.
- It decides where in the memory the program will be loaded.

2. Linking

- Combine diff. object Modules into a single program.
- Resolve external references (symbol defined in other modules).

3. Relocation

- Adjust address - dependent instrⁿ
- Modify address based on the actual memory location where the program is located.

4. Loading

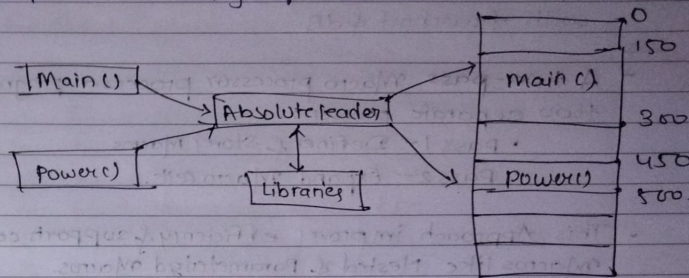
- Loads the program into main memory.
- Copies Instrⁿ & data from object file to RAM.

5. Execution

- Transfers control to the starting addresses the program
- Begins program execution

Object Program → Loader → Memory → Execution

Q8] Explain the design of Absolute loader.



- An absolute Loader is a type of Loader that loads the program into specific (fixed) memory location specified in the object program, without performing relocation or linking.

• Design of Absolute loader :

The design of absolute loader is simple bcz it directly places the object code into memory.

• Working steps :

1. The loader reads the object program.
2. It looks for starting address specified in the program.
3. Each instrⁿ is loaded into the exact memory location mentioned.
4. No modification of address is done.
5. After loading is complete, control is transferred to the starting execution address.

6

• Algo :

1. Read Header Record
2. Get starting address
3. Read text Record
4. Move object code to specified memory location.
5. Repeat Until End record.
6. Transfer control to Starting address.

Object prog → AL → main memory
↓
Executh.

Q7 Explain the working of two-pass Macro Processor with flowchart & DR.

- A two-pass Macro processor processes a program in two separate passes:
 - Pass 1 - Define & Store Macros
 - Pass 2 - Expand Macro calls.

- This Approach improves efficiency & supports complex Macros like Nested & Parameterized Macros.

Need for Two-Pass Macro Processor

- To separate macro defⁿ & Macro expansion.
- To handle forward references
- To manage nested macros effectively.

Pass 1: Macro Definition Processing

Function: Identify Macro defⁿ & Store them in tables.

Steps:

1. Read Source program line-by-line.
2. If Macro keyword found $\rightarrow$ start defⁿ.
3. Store Macro Name in MNT [Macro Name Table].
4. Store Macro body in MDT [Macro defⁿ Table].
5. Store parameter in ALA [Argument List Array].
6. Continued till MEND.
7. Non-Macro Statement = Copied to Intermediate file.

Pass 2: Macro Expansion

Function: Expand Macro calls using stored defⁿ.

Steps:

1. Read Intermediate Code.
2. Identify Macro call.
3. Search Macro name in MNT.
4. Fetch Macro body from MDT.
5. Substitute Actual Parameters using ALA.
6. Generate expanded Code.
7. Continue till end of program.

Databases Used:-

1. Macro Name Table (MNT)

Stores Macro names & pointer to MDT

Macro name	MDT index
INCR	1

2. Macro Defⁿ Table (MDT)

Stores Macro body

Index	Def ⁿ
1	ADD A1.

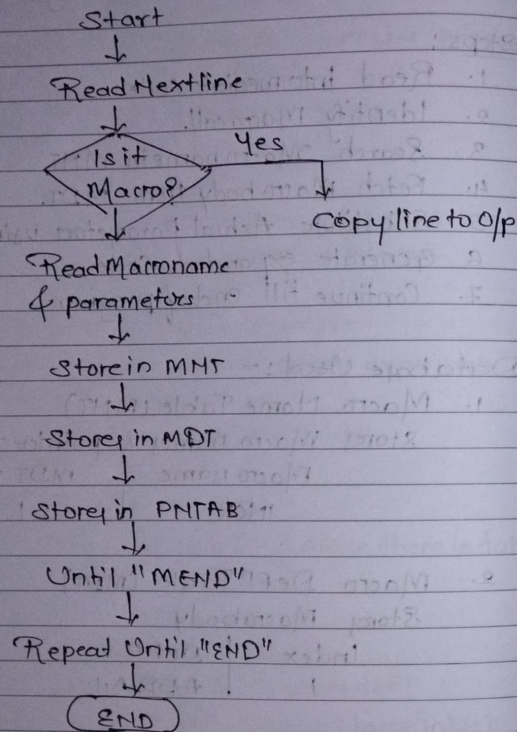
3. Argument List Array (ALA)

Maps parameter

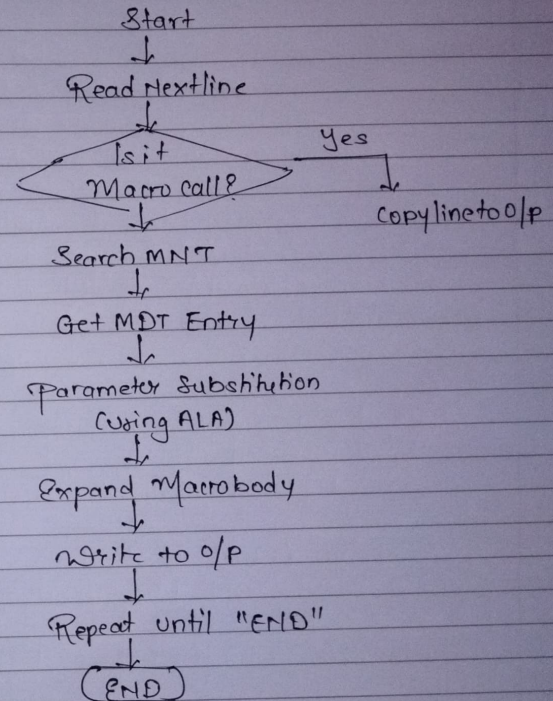
Formal $\rightarrow$ Actual

A $\rightarrow$ X.

Pass 1: Macro defn



Pass 2: Macro Expansion



Intermediate Code.

Need - $\begin{cases} \rightarrow \text{Source Code is Converted into Intermediate representation.} \\ \rightarrow \text{lies between Frontend \& Backend.} \end{cases}$

- It is a important Phase in a Compiler
- It Converts Source program into machine-independent intermediate form before final machine code.

Why it is needed.

1. Machine Independence -
Same intermediate codes can be used for diff. machines.
2. Easy Optimization -
Code can be optimized before converting to machine code.
3. Simplifies Compiler Design -
Breaks compilation into ~~4~~ phases.
4. ~~Port~~ Portability -
makes it easier to run the program on diff platforms.
5. Improves efficiency -
Helps generate better & faster target code.

Two types of Intermediate Code.

Three Address Code (TAC)

Uses simple instⁿ with at most 3 operands.
easy to translate into machine code.

ex - $a = b + c * d$

Converted To TAC $\rightarrow$ $t_1 = c * d$
 $t_2 = b + t_1$
 $a = t_2$

Syntax Tree (AST)

Tree representation of Program Structure.
each node represents a Operation &
Children represents operands.

ex = $a = b + c * d$

Tree Structure

